

DSAA3073: Theories in Data Science

Week 1: Introduction & Mathematical Preliminaries

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-27

Explorative projection

Opening Challenge

Why Do Learning Algorithms Work?

You train a classifier on 1000 examples and achieve 95% accuracy. You deploy it in production. What can you guarantee about its performance on new, unseen data?

- Can you bound the probability that test accuracy drops below 90%?
- How many more samples would you need to guarantee 99% confidence?
- Does the answer depend on the complexity of your model?

This week builds the mathematical foundation to answer these questions rigorously.

Introduction

Why do we need so much mathematics for machine learning? Here's the key insight: **learning from finite samples requires quantifying uncertainty**. We need probability to model randomness, geometry to structure our hypothesis spaces, and optimization to find good solutions efficiently.

Machine learning theory rests on three pillars: **probability** (to model uncertainty), **geometry** (to structure hypothesis spaces), and **optimization** (to find good solutions). This week establishes the essential tools from each domain.

Learning objectives: By the end of this week, you will:

1. Apply concentration inequalities to bound generalization error
2. Work with norms, inner products, and metric spaces
3. Recognize and exploit convexity in learning problems
4. Analyze computational complexity of algorithms

→ Week 2 Preview: PAC Learning

The tools we develop this week answer the opening question: Week 2 shows how Hoeffding's inequality and union bounds determine sample complexity for learning. You'll see exactly why 1000 samples might suffice—or why you might need 10,000.

Probability Theory Foundations

Probability Spaces and Random Variables

These formal definitions might seem abstract, but they're what allow us to make rigorous statements about learning from random samples. Think of training data as random draws from some unknown distribution—that's the **probability space** (defined below) in action.

Definition: Probability Space

A **probability space** is a triple $(\Omega, \mathcal{F}, \mathbb{P})$ where:

- Ω is the **sample space** (the set of all possible outcomes)
- $\mathcal{F}$ is a **σ -algebra** on Ω (a collection of measurable events—subsets of Ω to which we can assign probabilities)
- $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$ is a **probability measure** (a function assigning probabilities to events) with $\mathbb{P}(\Omega) = 1$

Concrete example: Rolling a die: $\Omega = \{1, 2, 3, 4, 5, 6\}$ (the six faces), $\mathcal{F}$ = all subsets of Ω (e.g., “roll is even” = $\{2, 4, 6\}$), and $\mathbb{P}$ assigns probability $\frac{1}{6}$ to each face.

Definition: Random Variable

A **random variable** is a measurable function $X : \Omega \rightarrow \mathbb{R}$ that maps outcomes to real numbers. When the integral exists, its **expectation** (average value) is

$$\mathbb{E}[X] = \int X \, d\mathbb{P} \quad (1)$$

and, when $\mathbb{E}[X^2] < \infty$, its **variance** (measure of spread around the mean) is $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$.

Concrete example: For a die roll, $X(\omega)$ = the number shown. Then $\mathbb{E}[X] = \frac{1+2+3+4+5+6}{6} = 3.5$ and $\text{Var}(X) = \mathbb{E}[(X - 3.5)^2] \approx 2.92$.

Concentration Inequalities

Here's where learning theory gets its power: **concentration inequalities** (mathematical bounds on how far random quantities deviate from their expected values) tell us that sample averages are close to their expectations with high probability. This is why we can trust training data to tell us about the whole population.

Concentration inequalities quantify how sample averages concentrate around their expectations—they answer questions like “with 1000 samples, how confident can I be that my estimate is within 5% of the true value?”

Markov's Inequality

Why start with Markov? It's the simplest concentration result, using only the mean. But watch how each subsequent inequality uses more information (variance, boundedness) to get exponentially tighter bounds.

Theorem: Markov's Inequality

For non-negative random variable X and $t > 0$,

$$\mathbb{P}(X \geq t) \leq \frac{\mathbb{E}[X]}{t} \quad (2)$$

In words: The probability that X is at least t times its mean cannot exceed $\frac{1}{t}$. For example, a non-negative variable cannot be 10 times its mean more than 10% of the time.

Proof (Guided Discovery):

Consider the **indicator function** $\mathbb{1}_{X \geq t}$, which equals 1 when $X \geq t$ and 0 otherwise. What can we say about its relationship to $\frac{X}{t}$?

Key insight: For non-negative X , we have $\mathbb{1}_{X \geq t} \leq \frac{X}{t}$. Why? When $X \geq t$, the indicator is 1 and $\frac{X}{t} \geq 1$, so $1 \leq \frac{X}{t}$. When $X < t$, the indicator is 0 and $\frac{X}{t} \geq 0$, so $0 \leq \frac{X}{t}$. In both cases, $\mathbb{1}_{X \geq t} \leq \frac{X}{t}$.

Now take expectations of both sides:

$$\mathbb{P}(X \geq t) = \mathbb{E}[\mathbb{1}_{X \geq t}] \leq \mathbb{E}\left[\frac{X}{t}\right] = \frac{\mathbb{E}[X]}{t} \quad (3)$$

Student Challenge

Exercise: Show that if X is bounded in $[0, M]$, Markov gives $\mathbb{P}(X \geq t) \leq \frac{M}{t}$ for $t > M$. Why is the exact probability actually 0?

Intuition: If the average is small, large values must be rare. But Markov is often loose because it only uses the first moment.

Chebyshev's Inequality

Chebyshev improves on Markov by using variance—the **second moment** (expected value of the squared variable, $\mathbb{E}[X^2]$). Notice the key trick: apply Markov to a squared deviation, which is always non-negative.

Theorem: Chebyshev's Inequality

For random variable X with finite variance and $k > 0$,

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq k) \leq \frac{\text{Var}(X)}{k^2} \quad (4)$$

In words: The probability that X deviates from its mean by at least k is bounded by the variance divided by k^2 . This bound decreases as $\frac{1}{k^2}$ —quadratically faster than Markov's $\frac{1}{k}$.

Proof (Guided Discovery):

We want to bound the probability that X deviates from its mean. Consider the squared deviation $(X - \mathbb{E}[X])^2$.

Critical observation: This is a non-negative random variable!

What happens if we apply Markov's inequality to $(X - \mathbb{E}[X])^2$ with threshold k^2 ?

$$\begin{aligned}\mathbb{P}(|X - \mathbb{E}[X]| \geq k) &= \mathbb{P}((X - \mathbb{E}[X])^2 \geq k^2) \\ &\leq \frac{\mathbb{E}[(X - \mathbb{E}[X])^2]}{k^2} \\ &= \frac{\text{Var}(X)}{k^2}\end{aligned}\tag{5}$$

Key Takeaway

Chebyshev improves on Markov by using the second moment (variance). The bound decays as $\frac{1}{k^2}$ instead of $\frac{1}{k}$.

✓ Checkpoint

Can you explain why Chebyshev's bound decays faster than Markov's as the deviation k increases? What information does variance capture that the mean alone doesn't?

Hoeffding's Inequality

This is the workhorse of learning theory. Hoeffding gives **exponential concentration** (bounds that decay exponentially with sample size)—the probability of large deviations decays like $\exp(-n\varepsilon^2)$, dramatically faster than Chebyshev's polynomial $\frac{1}{n\varepsilon^2}$. This exponential decay is why thousands of samples suffice for learning, not millions.

Theorem: Hoeffding's Inequality

Let $X_1, \dots, X_n$ be independent random variables with $X_i \in [a_i, b_i]$ (bounded). Then for any $\varepsilon > 0$,

$$\mathbb{P}\left(\left|\frac{1}{n} \sum_{i=1}^n X_i - \frac{1}{n} \sum_{i=1}^n \mathbb{E}[X_i]\right| \geq \varepsilon\right) \leq 2 \exp\left(-\frac{2n^2\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2}\right)\tag{6}$$

For **i.i.d.** (independent and identically distributed) variables in $[a, b]$, this simplifies to:

$$\mathbb{P}(|\bar{X}_n - \mathbb{E}[X]| \geq \varepsilon) \leq 2 \exp\left(-\frac{2n\varepsilon^2}{(b-a)^2}\right)\tag{7}$$

Here $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ is the **sample mean** (average of the observations).

In words: The probability that the sample average deviates from the true mean by more than ε decreases exponentially with the number of samples n and the squared tolerance ε^2 .

Proof Sketch (Key Ideas):

The full proof requires several technical steps. Here's the roadmap:

1. **One-sided tail:** First bound $\mathbb{P}\left(\sum_i (X_i - \mathbb{E}[X_i]) \geq n\varepsilon\right)$; the lower tail will be handled similarly at the end.
2. **Exponential moment method:** For any $t > 0$,

$$\mathbb{P}\left(\sum_i (X_i - \mathbb{E}[X_i]) \geq n\varepsilon\right) = \mathbb{P}\left(\exp\left(t \sum_i (X_i - \mathbb{E}[X_i])\right) \geq \exp(tn\varepsilon)\right) \quad (8)$$

3. **Apply Markov:**

$$\mathbb{P}\left(\exp\left(t \sum_i (X_i - \mathbb{E}[X_i])\right) \geq \exp(tn\varepsilon)\right) \leq \frac{\mathbb{E}\left[\exp\left(t \sum_i (X_i - \mathbb{E}[X_i])\right)\right]}{\exp(tn\varepsilon)} \quad (9)$$

4. **Independence:**

$$\mathbb{E}\left[\exp\left(t \sum_i (X_i - \mathbb{E}[X_i])\right)\right] = \prod_{i=1}^n \mathbb{E}[\exp(t(X_i - \mathbb{E}[X_i]))] \quad (10)$$

5. **Hoeffding's Lemma:** For $X \in [a, b]$ with $\mathbb{E}[X] = 0$,

$$\mathbb{E}[\exp(tX)] \leq \exp\left(t^2 \frac{(b-a)^2}{8}\right) \quad (11)$$

Proof idea: Use convexity of $\exp$ and the fact that X is bounded.

6. **Combine and optimize:**

$$\mathbb{P}\left(\sum_i (X_i - \mathbb{E}[X_i]) \geq n\varepsilon\right) \leq \exp\left(-tn\varepsilon + \frac{t^2 \sum_i (b_i - a_i)^2}{8}\right) \quad (12)$$

Minimize over t by setting derivative to zero: $t^* = \frac{4n\varepsilon}{\sum_i (b_i - a_i)^2}$

7. **Union bound:** Apply to both $\mathbb{P}(\bar{X}_n - \mathbb{E}[X] \geq \varepsilon)$ and $\mathbb{P}(\bar{X}_n - \mathbb{E}[X] \leq -\varepsilon)$

Student Challenge

Exercise: Verify that substituting t^* gives the stated bound. What is the decay rate in n ?

Proof Template: Exponential Moment Method

This technique appears throughout learning theory (Hoeffding, Chernoff, martingales). The pattern:

1. Transform tail probability: $\mathbb{P}(X \geq a) = \mathbb{P}(\exp(tX) \geq \exp(ta))$ for $t > 0$
2. Apply Markov to the exponential: $\mathbb{P}(\exp(tX) \geq \exp(ta)) \leq \frac{\mathbb{E}[\exp(tX)]}{\exp(ta)}$
3. Bound the moment generating function $\mathbb{E}[\exp(tX)]$ using structure (independence, boundedness)
4. Optimize over free parameter t
5. Union bound for two-sided inequalities

You'll see this pattern again in: Week 2 (PAC bounds), Week 10 (regret bounds)

Key Takeaway

Hoeffding provides **exponential concentration**: the probability of deviation decays as $\exp(-n\varepsilon^2)$. This is dramatically stronger than Chebyshev's $\frac{1}{n\varepsilon^2}$ bound. Moreover, it's **distribution-free**—only boundedness matters!

Example: Estimating Test Accuracy from Validation Set

Suppose you have a classifier and a validation set of $n = 1000$ labeled examples. Each prediction is correct (1) or incorrect (0), so $X_i \in \{0, 1\}$ are bounded in $[0, 1]$.

Your validation accuracy is $\hat{p} = \left(\frac{1}{n}\right) \sum_{i=1}^n X_i$. The true test accuracy is $p = \mathbb{E}[X]$.

By Hoeffding: $\mathbb{P}(|\hat{p} - p| \geq \varepsilon) \leq 2 \exp(-2 \cdot 1000 \cdot \varepsilon^2)$

For $\varepsilon = 0.05$ (5% error): $\mathbb{P}(|\hat{p} - p| \geq 0.05) \leq 2 \exp(-100) \approx 10^{-43}$

This is why 1000 samples suffice! With overwhelming probability, your validation accuracy is within 5% of the true test accuracy. This exponential concentration is the mathematical foundation of empirical validation.

→ Week 2 Preview: PAC Sample Complexity

Week 2 extends this reasoning to **hypothesis classes**. Instead of bounding one fixed classifier's error, we'll need to bound error uniformly over all hypotheses in a class. The union bound plus Hoeffding gives us PAC sample complexity bounds.

→ Week 4 Preview: Cross-Validation Analysis

Week 4 uses Hoeffding to analyze cross-validation: why does k-fold CV give reliable estimates of generalization error? The answer lies in these concentration inequalities.

Mental Model: Concentration Hierarchy

Think of concentration inequalities as a hierarchy of tightness:

Markov: $\text{Prob}(|X - \mu| \geq k) \leq 1/k$ (uses: mean only)

↓ tighter (uses variance)

Chebyshev: $\text{Prob}(|X - \mu| \geq k) \leq \sigma^2/k^2$ (uses: mean + variance)

↓ much tighter (uses boundedness + independence)

Hoeffding: $\text{Prob}(|\bar{X} - \mu| \geq \varepsilon) \leq 2e^{(-2n\varepsilon^2)}$ (uses: bounds + independence)

Decay rates matter: For $n = 1000, \varepsilon = 0.1$:

- Markov (loose, rarely used): bound ≈ 0.1
- Chebyshev: bound $\approx 1/(100\varepsilon^2) = 0.01$ (assuming $\sigma = 1$)
- Hoeffding: bound $\approx 10^{-8}$ (exponentially small!)

This hierarchy explains why Hoeffding dominates in learning theory.

 Theorem Landscape
Concentration Inequalities Roadmap:

We've seen three results in order of increasing power:

1. **Markov** (simplest): Uses only non-negativity and mean. Bound: $O(\frac{1}{t})$. Loose but very general.
2. **Chebyshev** (polynomial): Applies Markov to squared deviations. Bound: $O(\frac{1}{k^2})$. Requires finite variance. Applies to any distribution.
3. **Hoeffding** (exponential): Uses independence and boundedness. Bound: $O(\exp(-n\varepsilon^2))$. Distribution-free but requires i.i.d. samples.

Why this progression? Each uses more structure to get tighter bounds. Markov $\rightarrow$ Chebyshev is a clever application trick. Chebyshev $\rightarrow$ Hoeffding requires the exponential moment method (a more sophisticated technique).

When to use which? In learning theory, almost always Hoeffding (or relatives like Chernoff, Bernstein, McDiarmid). The exponential decay is what makes PAC learning tractable.

Inequality	Assumption	Uses	Tail behavior
Markov	nonnegative X	mean	$\frac{1}{t}$
Chebyshev	finite variance	variance	$\frac{1}{k^2}$
Hoeffding	independent bounded variables	bounds $[a_i, b_i]$	$\exp(-n\varepsilon^2)$

Progress on backbone: Hoeffding's inequality is the mathematical foundation for generalization bounds in PAC learning (Week 2). The exponential decay $\exp(-n\varepsilon^2)$ is what allows us to make finite-sample guarantees: with enough data, we can be confident that training error approximates test error. This directly answers our opening challenge about deployment guarantees.

Weak Law of Large Numbers

The Law of Large Numbers is foundational: sample averages converge to expectations as $n \rightarrow \infty$. But notice the limitation—it's asymptotic. We don't know **how large** n must be. That's why learning theory relies on Hoeffding, which gives explicit finite-sample bounds.

Theorem: Weak Law of Large Numbers

Let $X_1, \dots, X_n$ be i.i.d. with $\mathbb{E}[X] = \mu$ and $\text{Var}(X) = \sigma^2 < \infty$. Then for any $\varepsilon > 0$,

$$\mathbb{P}(|\bar{X}_n - \mu| \geq \varepsilon) \rightarrow 0 \text{ as } n \rightarrow \infty \quad (13)$$

Proof (Guided Discovery):

What are the expectation and variance of the sample mean $\bar{X}_n = (\frac{1}{n}) \sum_{i=1}^n X_i$?

By linearity of expectation:

$$\mathbb{E}[\bar{X}_n] = \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n X_i\right] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[X_i] = \mu \quad (14)$$

For variance, using independence:

$$\text{Var}(\bar{X}_n) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n} \quad (15)$$

Key insight: Variance decreases as $\frac{1}{n}$!

Now apply Chebyshev's inequality:

$$\mathbb{P}(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\text{Var}(\bar{X}_n)}{\varepsilon^2} = \frac{\sigma^2}{n\varepsilon^2} \rightarrow 0 \quad (16)$$

Connection

ML Interpretation: The empirical risk (average loss on training data) converges to the true risk (expected loss). This justifies using training error as a proxy for test error—but only asymptotically!

✓ Checkpoint

The WLLN says convergence happens “as $n \rightarrow \infty$ ”, but in practice we have finite n . How many samples do we actually need for the sample mean to be within ε of the true mean with probability $1 - \delta$? (Hint: Use Chebyshev or Hoeffding to get an explicit bound on n .)

Central Limit Theorem

The CLT is beautiful—it says that sums converge to Gaussians regardless of the original distribution. But notice the key limitation for learning theory: it's asymptotic and doesn't give finite-sample bounds. That's why we rely on Hoeffding for PAC learning, not the CLT.

Theorem: Central Limit Theorem (Statement Only)

Let $X_1, \dots, X_n$ be i.i.d. with $\mathbb{E}[X] = \mu$ and $\text{Var}(X) = \sigma^2 < \infty$. Then

$$\frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma} \rightarrow \mathcal{N}(0, 1) \quad \text{in distribution} \quad (17)$$

Intuition: Regardless of the original distribution, normalized sums become Gaussian. This is remarkable—flip coins, roll dice, measure heights—the sum always looks Gaussian for large n .

Limitation: CLT is **asymptotic**. It doesn't tell us how large n must be, nor does it give finite-sample bounds. For learning theory, we need Hoeffding-type inequalities.

Progress on backbone: We now have the probability tools to quantify uncertainty and bound generalization error. But learning algorithms don't just work with probabilities—they operate on structured spaces. This brings us to geometry.

Linear Algebra and Geometry

Why does machine learning need linear algebra? Because hypothesis spaces are often vector spaces—neural networks map inputs through layers of linear transformations, SVMs find separating hyperplanes, and PCA projects data onto lower-dimensional subspaces. Norms measure distance, inner products measure similarity, and these geometric structures are what make learning tractable.

Vector Spaces and Norms

Norms give us a way to measure “size” and “distance” in abstract vector spaces. In ML, we use norms to measure prediction error (ℓ_2 **loss**, introduced in Section 2.1 below), enforce sparsity (ℓ_1 **regularization**—a penalty term that encourages many weights to be exactly zero), and define hypothesis classes (bounded norm balls—sets of all vectors with norm at most some constant).

Definition: Norm

A **norm** on vector space V is a function $\|\cdot\| : V \rightarrow \mathbb{R}_+$ (that maps vectors to non-negative real numbers) satisfying:

1. **Positivity:** $\|x\| \geq 0$ with equality if and only if $x = 0$ (the zero vector)
2. **Homogeneity:** $\|\alpha x\| = |\alpha| \|x\|$ for all scalars $\alpha \in \mathbb{R}$ (scaling a vector scales its norm)
3. **Triangle inequality:** $\|x + y\| \leq \|x\| + \|y\|$ (direct path is shortest)

Intuition: A norm is a way to assign a “length” or “size” to every vector, generalizing the notion of Euclidean length.

Example: ℓ_p Norms

For $x \in \mathbb{R}^n$ (a vector with n components) and $p \geq 1$:

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}} \quad (18)$$

This is the ℓ_p **norm** (pronounced “ell-p norm”). Special cases:

- $\|x\|_1 = \sum_{i=1}^n |x_i|$ (ℓ_1 **norm** or Manhattan distance—sum of absolute values)
- $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$ (ℓ_2 **norm** or Euclidean distance—straight-line distance)
- $\|x\|_\infty = \max_{i=1,\dots,n} |x_i|$ (ℓ_∞ **norm** or maximum norm—largest component in absolute value)

Concrete example: For $x = (3, -4)$:

- $\|x\|_1 = |3| + |-4| = 7$
- $\|x\|_2 = \sqrt{3^2 + 4^2} = 5$
- $\|x\|_\infty = \max\{3, 4\} = 4$

Example: Feature Normalization in ML

When training a neural network, we often normalize input features to have similar scales. Why?

Suppose feature 1 is income (range: \$0–\$200,000) and feature 2 is age (range: 0–100). Without normalization, gradient descent updates will be dominated by the income dimension because $|x_1|$ is $2000\times$ larger than $|x_2|$.

Solution: Normalize to $\|x\|_2 = 1$ or standardize to mean 0, variance 1. This makes gradient descent converge faster by equalizing the scale across dimensions.

Different norms capture different notions of size: ℓ_1 encourages sparsity (many zeros), ℓ_2 is rotation-invariant (Euclidean geometry), ℓ_∞ bounds the maximum component.

 Mental Model: ℓ_p Norm Unit Balls

Visualize the unit balls $\{x : \|x\|_p = 1\}$ in $\mathbb{R}^2$:

ℓ_1 ($p=1$): Diamond shape ($|x_1| + |x_2| = 1$)
Encourages sparsity – optimal points tend to lie on axes

ℓ_2 ($p=2$): Circle ($x_1^2 + x_2^2 = 1$)
Rotation-invariant – all directions treated equally

ℓ_∞ ($p=\infty$): Square ($\max\{|x_1|, |x_2|\} = 1$)
Bounds maximum coordinate

ML Application: Regularization penalizes $\|\theta\|_p$. The shape of the ball determines which solutions are favored:

- ℓ_1 : Sparse weights (feature selection)
- ℓ_2 : Small weights uniformly (ridge regression - Week 4)
- ℓ_∞ : Bounded maximum weight

Theorem: Equivalence of Norms in Finite Dimensions

In $\mathbb{R}^n$, all norms are equivalent: for any two norms $\|\cdot\|_a$ and $\|\cdot\|_b$, there exist constants $c, C > 0$ such that

$$c\|x\|_a \leq \|x\|_b \leq C\|x\|_a \text{ for all } x \in \mathbb{R}^n \quad (19)$$

Proof Sketch: This uses the standard finite-dimensional compactness fact that the unit sphere of any norm in $\mathbb{R}^n$ is compact (equivalently, all finite-dimensional norm topologies agree). The function $f(x) = \|x\|_b$ is continuous on $S = \{x : \|x\|_a = 1\}$, so it attains its minimum c and maximum C on S . By homogeneity, the inequality extends to all x .

Why this matters: In finite dimensions, convergence in one norm implies convergence in all norms. But in infinite dimensions (function spaces), this fails! This is why function space theory carefully specifies which norm to use.

✓ Checkpoint

Why does norm equivalence fail in infinite dimensions? Think about the sequence of functions $f_{n(x)} = x^n$ on $[0, 1]$. Does $\|f_n\|_\infty \rightarrow 0$? Does $\|f_n\|_1 \rightarrow 0$? What does this tell you?

Inner Products

Inner products generalize dot products and give us a way to measure angles and orthogonality (perpendicularity). In ML, inner products appear in similarity measures between data points, feature maps, and neural network activations.

Definition: Inner Product

An **inner product** on vector space V is a function $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{R}$ (that takes two vectors and returns a scalar) satisfying:

1. **Linearity:** $\langle \alpha x + \beta y, z \rangle = \alpha \langle x, z \rangle + \beta \langle y, z \rangle$ for scalars α, β
2. **Symmetry:** $\langle x, y \rangle = \langle y, x \rangle$ (order doesn't matter)
3. **Positive-definiteness:** $\langle x, x \rangle \geq 0$ with equality if and only if $x = 0$

Every inner product induces a norm: $\|x\| = \sqrt{\langle x, x \rangle}$ (the norm is the square root of the inner product with itself).

Concrete example: The standard dot product in $\mathbb{R}^n$ is an inner product: $\langle x, y \rangle = \sum_{i=1}^n x_i y_i$. For $x = (1, 2)$ and $y = (3, 4)$: $\langle x, y \rangle = 1 \cdot 3 + 2 \cdot 4 = 11$.

Cauchy-Schwarz Inequality

Cauchy-Schwarz is everywhere in learning theory—it bounds correlations, controls **Rademacher complexity** (a data-dependent measure of hypothesis class complexity, introduced in Week 3), and enables many geometric learning methods. The proof technique (discriminant argument on a quadratic) is a pattern you'll see repeatedly.

Theorem: Cauchy-Schwarz Inequality

For vectors x, y in an inner product space,

$$|\langle x, y \rangle| \leq \|x\| \|y\| \quad (20)$$

with equality if and only if x and y are linearly dependent (one is a scalar multiple of the other, or one is zero).

In words: The absolute value of the inner product of two vectors cannot exceed the product of their norms. This bounds how “aligned” two vectors can be.

Productive Struggle

Before reading the proof: Try to prove Cauchy-Schwarz yourself using this hint: consider $f(t) = \|x - ty\|^2$ and use the fact that $f(t) \geq 0$ for all $t \in \mathbb{R}$.

What does this constraint tell you about the discriminant of the quadratic $f(t)$?

Spend 5 minutes attempting this. The discriminant argument is a powerful technique that appears in many proofs (including convexity results later this week).

Proof (Guided Discovery):

If $y = 0$, the inequality is immediate. Otherwise, consider the function $f(t) = \|x - ty\|^2$ for $t \in \mathbb{R}$. What can we say about this function?

Key insight: Since $\|\cdot\|^2 \geq 0$, we have $f(t) \geq 0$ for all t .

Expand using the inner product:

$$\begin{aligned} f(t) &= \langle x - ty, x - ty \rangle \\ &= \langle x, x \rangle - 2t\langle x, y \rangle + t^2\langle y, y \rangle \\ &= \|x\|^2 - 2t\langle x, y \rangle + t^2\|y\|^2 \end{aligned} \tag{21}$$

This is a quadratic in t that is always non-negative. What does this tell us about its discriminant?

For a quadratic $at^2 + bt + c \geq 0$ for all t , we need $b^2 - 4ac \leq 0$.

Here: $a = \|y\|^2$, $b = -2\langle x, y \rangle$, $c = \|x\|^2$

$$4\langle x, y \rangle^2 - 4\|x\|^2\|y\|^2 \leq 0 \tag{22}$$

Therefore: $\langle x, y \rangle^2 \leq \|x\|^2\|y\|^2$

Student Challenge

Exercise: When does equality hold? Hint: The discriminant is zero when the quadratic has exactly one root.

Applications:

- Triangle inequality for induced norm
- Correlation bounds: $|\text{Corr}(X, Y)| \leq 1$
- Hölder's inequality (generalization to L^p spaces)

→ Week 3 Preview: Rademacher Complexity

Cauchy-Schwarz plays a key role in bounding Rademacher complexity (Week 3), which refines VC dimension bounds using data-dependent measures. The inequality controls how much a hypothesis class can correlate with random noise.

Matrix Decompositions

Singular Value Decomposition

SVD is the workhorse of numerical linear algebra. In ML, it enables **PCA** (Principal Component Analysis—a dimensionality reduction technique), low-rank matrix completion, and understanding neural network compression.

Theorem: Singular Value Decomposition

Every matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be written as

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (23)$$

where:

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ is an **orthogonal matrix** (columns are orthonormal vectors—perpendicular unit vectors)
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is a **diagonal matrix** (all entries zero except on the diagonal) with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ called the **singular values**

In words: Any matrix can be decomposed into a rotation, followed by axis-aligned scaling, followed by another rotation.

Geometric Interpretation: Any linear transformation $\mathbf{A}$ acts as: (1) rotate via $\mathbf{V}^T$, (2) scale along coordinate axes via $\mathbf{\Sigma}$, (3) rotate via $\mathbf{U}$. The singular values σ_i are the scaling factors along each axis.

Theorem: Eckart-Young Theorem

One best rank- k approximation to $\mathbf{A}$ in Frobenius norm is

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (24)$$

with error $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$ where $r = \text{rank}(\mathbf{A})$. If singular values are tied at the cutoff, the best approximation need not be unique.

Example: Dimensionality Reduction

For image compression: if singular values decay rapidly (e.g., $\sigma_k \approx 0$ for $k > 100$), we can store only the first 100 singular vectors instead of the full matrix, achieving massive compression with small error.

Example: Image Compression via Low-Rank Approximation

Consider a 1000×1000 grayscale image (1 million pixels). Store it as a matrix $\mathbf{A} \in \mathbb{R}^{1000 \times 1000}$.

Full storage: 1,000,000 values

SVD compression: Compute $\mathbf{A} \approx \mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ with $k = 50$

Storage required: 50 singular values + 50 left singular vectors (1000 each) + 50 right singular vectors (1000 each) = $50 + 50,000 + 50,000 = 100,050$ values

Compression ratio: $10\times$, with small perceptual error if singular values decay rapidly

Why it works: Natural images have structure (smooth regions, edges), so rank is effectively low. Random noise would have all singular values equal—no compression possible.

→ Week 11 Preview: Neural Network Compression

Week 11 discusses how SVD compresses neural network weight matrices. Large models can often be approximated with low-rank matrices, reducing parameters while maintaining accuracy.

✓ Checkpoint

Suppose all singular values of a 1000×1000 matrix are equal: $\sigma_1 = \sigma_2 = \dots = \sigma_{1000}$. Can you achieve compression via low-rank approximation? Why or why not? What does this tell you about when SVD compression works?

Progress on backbone: Linear algebra gave us finite-dimensional tools—vectors, matrices, norms. But many hypothesis classes are infinite-dimensional function spaces. This raises the question: how do we extend geometric reasoning to infinity?

Functional Analysis Basics

Functional analysis extends linear algebra to infinite dimensions. Why does ML need this? Because hypothesis classes are often function spaces—neural networks approximate functions, and convergence analysis requires understanding limits in function spaces.

Metric Spaces

Metric spaces formalize the notion of “distance” between objects. In ML, we measure distance between data points (for clustering), between hypotheses (for regularization), and between distributions (for generative models). The **triangle inequality** (defined below) is what makes gradient descent work—moving toward the optimum in small steps eventually gets you there.

Definition: Metric Space

A **metric space** is a pair (X, d) where X is a set and $d : X \times X \rightarrow \mathbb{R}_+$ is a **distance function** satisfying:

1. **Identity:** $d(x, y) = 0$ if and only if $x = y$
2. **Symmetry:** $d(x, y) = d(y, x)$
3. **Triangle inequality:** $d(x, z) \leq d(x, y) + d(y, z)$

A metric space is **complete** if every Cauchy sequence (where points get arbitrarily close) converges to a point in the space.

Why completeness matters: Completeness means “no holes”—crucial for guaranteeing convergence of iterative optimization algorithms.

Function Spaces

These are the spaces where learning theory happens. **Banach spaces** are complete normed vector spaces. **Hilbert spaces** are complete inner product spaces. Understanding these spaces is essential for analyzing function approximation in learning algorithms.

Definition: Banach and Hilbert Spaces

A **Banach space** is a complete normed vector space. A **Hilbert space** is a complete inner product space.

Key examples: $\mathbb{R}^n$ with any norm is Banach; $\mathbb{R}^n$ with inner product is Hilbert; $L^2(X) = \{f : \int |f|^2 < \infty\}$ (square-integrable functions) is an infinite-dimensional Hilbert space.

Relationship: Every Hilbert space is Banach (inner products induce norms), but not vice versa.

Connection

Key intuition: Hilbert spaces let us do geometry with functions. Completeness guarantees that Cauchy sequences from iterative algorithms converge to points in the space.

Progress on backbone: We can now reason about convergence in function spaces. But having a space of hypotheses isn't enough—we need to **find** good ones. This motivates optimization: the computational engine that powers learning.

Optimization Fundamentals

Optimization is how we find good hypotheses—minimize training error, maximize margin, or minimize a regularized loss. Convexity is the holy grail: it guarantees that gradient descent finds the global optimum. Understanding when problems are convex (and when they're not) is crucial for designing learning algorithms.

Convexity

Convexity is rare in the real world but common by design in ML. We often formulate problems as **convex optimization** (optimization where both the objective function and constraint sets are convex) because convex problems have no local minima—gradient descent reliably finds the global optimum. SVMs, logistic regression, and ridge regression are all convex. Neural networks? Not convex—which is why training them is harder.

Definition: Convex Set

A set $C \subseteq \mathbb{R}^n$ is **convex** if for all $x, y \in C$ and $\lambda \in [0, 1]$,

$$\lambda x + (1 - \lambda)y \in C \quad (25)$$

In words: The line segment connecting any two points in the set stays entirely within the set. Think of convex sets as having “no dents” or “no holes.”

Examples: Balls, half-spaces, and intersections of convex sets are convex. A crescent moon shape is not convex (the line between two points on opposite tips goes outside).

Intuition: A convex set has no “dents.” If you can see two points, you can travel between them in a straight line without leaving the set.

Definition: Convex Function

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **convex** if its domain is convex and for all x, y in the domain and $\lambda \in [0, 1]$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y) \quad (26)$$

In words: The function value at any point on a line segment is at most the linear interpolation of the function values at the endpoints. Graphically, the function “curves upward”—any chord lies above the graph.

Visual intuition: If you draw a line between any two points on the graph of f , the function stays below that line.

Example: Why Logistic Regression Works

Logistic regression minimizes the log-loss: $f(\theta) = \sum_{i=1}^n \log(1 + \exp(-y_i \theta^T x_i))$

Key fact: $f(\theta)$ is convex in θ . Why?

- The function $z \mapsto \log(1 + e^z)$ is convex (second derivative positive)
- Composition with linear maps preserves convexity: $\theta \mapsto -y_i \theta^T x_i$ is linear

Consequence: Gradient descent on logistic regression finds the global minimum. No local minima to worry about. This is why logistic regression is so reliable compared to neural networks, where non-convexity creates optimization challenges.

→ Week 4 Preview: Ridge Regression Analysis

Week 4 analyzes ridge regression: $\min_{\theta} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2$. Both terms are convex, so their sum is convex. This guarantees a unique global minimum with closed-form solution $\theta^* = (X^T X + \lambda I)^{-1} X^T y$.

Theorem: First-Order Characterization

A differentiable function f on a convex open domain $D \subseteq \mathbb{R}^n$ is convex iff

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) \text{ for all } x, y \in D \quad (27)$$

▣ Proof Template: First-Order Characterization of Convexity

This characterization says: “The tangent line at any point is a global underestimator.” It’s a pattern that appears throughout optimization theory.

Proof technique:

1. ($\Rightarrow$) Assume convexity, consider $g(t) = f(x + t(y - x))$, apply convexity to g
2. Take $t \rightarrow 0$ to extract the first-order condition
3. ($\Leftarrow$) Assume first-order condition, apply it at $z = \lambda x + (1 - \lambda)y$
4. Take convex combination to recover definition of convexity

You’ll see this technique in: Week 10 (gradient descent convergence), convex analysis throughout

Key insight: For convex f , the linearization at any point underestimates f everywhere. This is why gradient descent makes progress—moving in the negative gradient direction decreases the function value.

Proof (Guided Discovery):

($\Rightarrow$) Assume f is convex. Fix x, y and consider $g(t) = f(x + t(y - x))$ for $t \in [0, 1]$.

By convexity:

$$g(t) = f(x + t(y - x)) \leq (1 - t)f(x) + tf(y) \quad (28)$$

Rearrange:

$$\frac{g(t) - g(0)}{t} \leq f(y) - f(x) \quad (29)$$

Take $t \rightarrow 0^+$:

$$g'(0) = \nabla f(x)^T(y - x) \leq f(y) - f(x) \quad (30)$$

($\Leftarrow$) Assume the first-order condition holds. For x, y and $\lambda \in [0, 1]$, let $z = \lambda x + (1 - \lambda)y$.

Apply the condition at z :

$$\begin{aligned} f(x) &\geq f(z) + \nabla f(z)^T(x - z) \\ f(y) &\geq f(z) + \nabla f(z)^T(y - z) \end{aligned} \quad (31)$$

Multiply the first by λ , the second by $(1 - \lambda)$, and add:

$$\lambda f(x) + (1 - \lambda)f(y) \geq f(z) + \nabla f(z)^T(\lambda(x - z) + (1 - \lambda)(y - z)) = f(z) \quad (32)$$

Student Challenge

Exercise: Verify the last equality. Why does the gradient term vanish?

Theorem: Second-Order Characterization

A twice-differentiable function f on a convex open domain $D \subseteq \mathbb{R}^n$ is convex iff its Hessian $\nabla^2 f(x)$ is positive semidefinite for all $x \in D$.

Proof Sketch: Restrict f to any line segment in D : $g(t) = f(x + t(y - x))$. Convexity of f is equivalent to convexity of every such one-dimensional restriction, and $g''(t) = (y - x)^T \nabla^2 f(x + t(y - x))(y - x)$. Thus positive semidefiniteness of the Hessian gives nonnegative second derivative along every line.

Mental Model: Convex vs Nonconvex Landscapes

Visualize optimization landscapes:

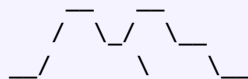
Convex function:



One global minimum

Gradient descent works

Nonconvex function:



Multiple local minima

May get stuck in local minima

ML Examples:

- **Convex:** Linear regression, logistic regression, SVM, ridge/lasso
- **Nonconvex:** Neural networks, matrix factorization, most deep learning

Why nonconvex is harder: Gradient descent may converge to a local minimum that's far from optimal. Initialization matters. Learning rate tuning is critical.

Week 10 preview: Even for nonconvex problems, understanding convex optimization helps. Many nonconvex problems have convex relaxations, or local convexity properties we can exploit.

Theorem Landscape

Convexity Characterizations:

We have three equivalent ways to check convexity for smooth functions:

1. **Definition:** $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$ (geometric: function lies below chords)
2. **First-order:** $f(y) \geq f(x) + \nabla f(x)^T(y - x)$ (tangent line underestimates everywhere)
3. **Second-order:** $\nabla^2 f(x) \geq 0$ (Hessian is positive semidefinite)

Why have three? Different characterizations are useful in different contexts:

- Definition: Conceptual understanding, proving properties
- First-order: Analyzing gradient descent, proving convergence
- Second-order: Checking convexity of specific functions (just compute Hessian eigenvalues)

Relationship: Definition $\Rightarrow$ First-order (via calculus) $\Rightarrow$ Second-order (differentiate again). All are equivalent for twice-differentiable functions.

Jensen's Inequality

Jensen's inequality connects convexity to probability—it says that for convex functions, the expectation of the function is at least the function of the expectation. This appears in information theory, variational inference, and bounding generalization error.

Theorem: Jensen's Inequality

For a differentiable convex function φ and an integrable random variable X with $\mathbb{E}[|\varphi(X)|] < \infty$,

$$\varphi(\mathbb{E}[X]) \leq \mathbb{E}[\varphi(X)] \quad (33)$$

(Reverse for concave φ .)

Proof (Guided Discovery):

By the first-order characterization, for any a :

$$\varphi(x) \geq \varphi(a) + \varphi'(a)(x - a) \quad (34)$$

What happens if we set $a = \mathbb{E}[X]$ and take expectations?

$$\mathbb{E}[\varphi(X)] \geq \varphi(\mathbb{E}[X]) + \varphi'(\mathbb{E}[X])(\mathbb{E}[X] - \mathbb{E}[X]) = \varphi(\mathbb{E}[X]) \quad (35)$$

Applications:

- AM-GM inequality: $\sqrt{xy} \leq \frac{x+y}{2}$ (from concavity of $\log$)
- Information theory: entropy is concave
- Variational inference: ELBO derivation

Gradients and Optimality

Here's the key insight: for convex functions, any local minimum is a global minimum. And we can find it by setting the gradient to zero. This is why convex optimization is tractable.

Definition: Gradient

The **gradient** of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ at x is

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]^T \quad (36)$$

It points in the direction of steepest ascent.

Theorem: Optimality for Convex Functions

Let f be differentiable and convex on a convex open domain $D \subseteq \mathbb{R}^n$. For an interior point $x^* \in D$, x^* is a global minimum iff $\nabla f(x^*) = 0$.

Proof: By first-order characterization, $\nabla f(x^*) = 0$ implies $f(y) \geq f(x^*)$ for all $y \in D$. Conversely, if an interior point x^* is a minimizer of differentiable f , then the directional derivative in every direction is zero, so $\nabla f(x^*) = 0$.

Key Takeaway

Power of Convexity: Any local minimum of a convex function is global, and under smoothness/step-size assumptions gradient methods can be analyzed cleanly. This is why many ML problems are formulated as convex optimization (linear regression, SVM, logistic regression).

→ **Week 10 Preview: Gradient Descent Convergence**

Week 10 analyzes gradient descent for convex functions: $\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$. For strongly convex f , gradient descent converges at rate $O(\exp(-t))$ (exponentially fast!). For general convex f , convergence is $O(\frac{1}{t})$ (polynomial).

✓ **Checkpoint**

Consider training a neural network (nonconvex) vs. logistic regression (convex). Both use gradient descent with the same learning rate. Which is more likely to find the global optimum? Which is more sensitive to initialization? Which needs more careful hyperparameter tuning? Explain using the convexity concepts from this section.

Constrained Optimization

The KKT conditions are the centerpiece of constrained optimization. They tell us exactly when a point is optimal for problems with both equality and inequality constraints.

Theorem: KKT Conditions

Consider the convex optimization problem:

$$\min_x f(x) \text{ subject to } g_i(x) \leq 0, h_j(x) = 0 \quad (37)$$

Assume f and g_i are differentiable convex functions, the equality constraints h_j are affine, and a constraint qualification holds (for example, Slater's condition for the inequality constraints). Then x^* is optimal iff there exist $\lambda_i \geq 0, \nu_j$ such that:

1. **Stationarity:** $\nabla f(x^*) + \sum_i \lambda_i \nabla g_i(x^*) + \sum_j \nu_j \nabla h_j(x^*) = 0$
2. **Primal feasibility:** $g_i(x^*) \leq 0, h_j(x^*) = 0$
3. **Dual feasibility:** $\lambda_i \geq 0$
4. **Complementary slackness:** $\lambda_i g_i(x^*) = 0$

Without a constraint qualification, KKT conditions remain sufficient for convex problems but may fail to be necessary. In other words, convexity makes KKT a certificate of optimality, while the constraint qualification ensures every optimum has such a certificate.

Intuition: At the optimum, the gradient of the objective is a linear combination of constraint gradients. Inactive constraints ($g_i(x^*) < 0$) have $\lambda_i = 0$.

 Confidence Check

Rate your understanding (1-5):

- ☐ I can explain why convex problems are easier to optimize than nonconvex problems
- ☐ I can use the first-order characterization to prove a function is convex
- ☐ I understand when gradient descent is guaranteed to find the global optimum
- ☐ I can explain the role of KKT conditions in constrained optimization

If you rated yourself below 3 on any item, review that section before continuing. These convexity concepts are foundational for Weeks 4, 7, and 10.

Progress on backbone: Optimization tells us **how** to find good hypotheses algorithmically. But can we find them **efficiently**? Not all optimization problems are tractable. This brings us to computational complexity—the theory that distinguishes feasible from infeasible.

Complexity Theory Basics

Computational complexity tells us what's possible in reasonable time. Many learning problems are provably hard (NP-complete), yet we solve them every day. Understanding complexity helps us know when to expect exact solutions, when to use approximations, and why heuristics often work well despite worst-case hardness.

Asymptotic Notation

Big-O notation abstracts away constant factors and focuses on growth rates. When we say an algorithm is $O(n^2)$, we mean it scales quadratically—doubling the input size quadruples the runtime (approximately). This is the language of scalability.

Definition: Big-O Notation

- $f(n) = O(g(n))$ if $\exists c, n_0 : f(n) \leq cg(n)$ for all $n \geq n_0$
- $f(n) = \Omega(g(n))$ if $g(n) = O(f(n))$
- $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$
- $f(n) = o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

Intuition: Big-O describes growth rate up to constant factors. It's the language of algorithm analysis.

Computational Complexity Classes

Here's the fundamental tension in ML: many learning problems are NP-hard (finding the optimal decision tree, selecting the best features, training neural networks to global optimality). Yet we train models every day. How? We use approximations, heuristics, and convex relaxations that work well in practice despite worst-case hardness.

Definition: Complexity Classes

- **P:** Problems solvable in polynomial time
- **NP:** Problems whose solutions are verifiable in polynomial time
- **NP-hard:** At least as hard as the hardest NP problems
- **NP-complete:** NP-hard and in NP

ML Relevance: Many learning problems are NP-hard:

- Feature selection (subset selection)
- Training neural networks (finding global minimum)
- Learning decision trees (optimal tree)

Yet we solve them in practice! How?

- Approximation algorithms
- Heuristics (gradient descent, greedy methods)

- Convex relaxations
- Randomization

Key Takeaway

Theory guides practice, but computational intractability doesn't mean practical impossibility. Understanding complexity helps us design better algorithms and set realistic expectations.

✓ Checkpoint

Training a neural network to find the global minimum is NP-hard. Yet we train neural networks successfully all the time. Explain this apparent contradiction. What are we actually finding when we train with gradient descent?

Integration and Preview

Let's connect the dots: we now have all the mathematical tools needed to build learning theory. Probability models uncertainty, concentration bounds generalization, geometry structures hypothesis spaces, optimization finds solutions, and complexity tells us what's tractable.

Synthesis

We now have the tools to formalize machine learning:

1. **Probability:** Model data as random samples from a distribution
2. **Concentration:** Bound generalization error (training vs. test performance)
3. **Geometry:** Structure hypothesis spaces with norms and metrics
4. **Optimization:** Find good hypotheses via gradient descent
5. **Complexity:** Assess computational feasibility

Answering the opening challenge: Remember the deployment question—can we guarantee test accuracy stays above 90% with 99% confidence? Using Hoeffding's inequality with the tools from this week, we can now compute exact sample size requirements: $n \geq \frac{1}{2\varepsilon^2} \log \frac{2}{\delta}$ where $\varepsilon = 0.05$ (the 5% accuracy drop) and $\delta = 0.01$ (99% confidence). This mathematical foundation transforms vague hopes into rigorous guarantees.

Preview of Week 2-3: PAC Learning

Here's where everything comes together. Week 2 uses Hoeffding plus union bounds to derive sample complexity. The key question: how many samples do we need for learning to be possible?

The PAC Framework (agnostic bounded-loss preview): For losses in $[0, 1]$, a hypothesis class $\mathcal{H}$ is learnable if there exists an algorithm that, given $\varepsilon, \delta > 0$ and enough iid samples $n \geq m_{\mathcal{H}}(\varepsilon, \delta)$, outputs $h \in \mathcal{H}$ such that, with probability at least $1 - \delta$ over the training sample (and any algorithmic randomness),

$$R(h) - \min_{h' \in \mathcal{H}} R(h') \leq \varepsilon \quad (38)$$

Key Questions:

- How does $m_{\mathcal{H}}(\varepsilon, \delta)$ depend on $\mathcal{H}$? (VC dimension)
- Can we bound $R(h) - \hat{R}(h)$ uniformly over $\mathcal{H}$? (Rademacher complexity)
- What's the computational cost? (Efficient PAC learning)

Hoeffding's inequality gives us the first step: for a **fixed** hypothesis h ,

$$\mathbb{P}(|R(h) - \hat{R}(h)| \geq \varepsilon) \leq 2 \exp(-2n\varepsilon^2) \quad (39)$$

But we need **uniform** convergence over all $h \in \mathcal{H}$. This requires measuring the “size” of $\mathcal{H}$ —enter VC dimension!

Key Takeaway

The Big Picture: Machine learning works because:

1. Sample averages concentrate (Hoeffding)
2. Hypothesis classes have finite complexity (VC dimension)
3. Optimization finds good hypotheses (convexity or heuristics)
4. Algorithms run in reasonable time (polynomial complexity)

Week 1 provides the foundation. Weeks 2-12 build the theory.

Construct Your Own

Design a Simple PAC Scenario:

Create a concrete learning problem and analyze its sample complexity using Week 1 tools:

1. Choose a simple hypothesis class (e.g., thresholds on the real line, axis-aligned rectangles in $\mathbb{R}^2$)
2. Define a loss function (e.g., 0-1 loss)
3. Use Hoeffding to bound the probability that one fixed hypothesis has error $|R(h) - \hat{R}(h)| > \varepsilon$
4. If your hypothesis class is finite with cardinality $|\mathcal{H}|$, use union bound: $\mathbb{P}(\exists h : |R(h) - \hat{R}(h)| > \varepsilon) \leq |\mathcal{H}| \cdot 2 \exp(-2n\varepsilon^2)$
5. Solve for n to make this probability at most δ

Goal: Derive a concrete bound like “For thresholds on $[0, 1]$ with 0-1 loss, $n \geq \frac{1}{2\varepsilon^2} (\log|\mathcal{H}| + \log(\frac{2}{\delta}))$ samples suffice for PAC learning.”

This exercise previews Week 2’s main results!

 Synthesis Challenge**Connecting Tools to the Learning Pipeline:**

For each Week 1 tool, identify where it appears in the learning pipeline:

1. **Hoeffding's inequality:** Where does exponential concentration appear in bounding generalization error?
2. **Cauchy-Schwarz:** How does it bound correlations in Rademacher complexity (Week 3)?
3. **Convexity:** Which ML problems are convex? Which are not? What's the impact on optimization?
4. **Union bound:** How does it extend Hoeffding from one hypothesis to a class of hypotheses?
5. **Big-O notation:** Express the sample complexity you derived in the construct-your-own exercise using asymptotic notation.

Goal: See how Week 1's abstract theorems become concrete tools for analyzing learning algorithms.

Key Takeaways

Let's consolidate what we've learned. Each of these tools will reappear throughout the course—this is your mathematical foundation for learning theory.

1. **Concentration inequalities** (Markov, Chebyshev, Hoeffding) quantify how sample statistics concentrate around population parameters. Hoeffding's exponential decay is the workhorse of learning theory.
2. **Norms and inner products** provide geometric structure. Cauchy-Schwarz is ubiquitous. All norms are equivalent in finite dimensions, but not in infinite dimensions.
3. **Convexity** is rare but powerful. Convex problems have no local minima—gradient descent finds the global optimum. First-order condition: tangent line is global underestimator.
4. **Completeness** (Banach/Hilbert spaces) is one ingredient in convergence analysis: once an argument shows a sequence is Cauchy, completeness ensures the limit is still in the space.
5. **Computational complexity** matters. Many learning problems are NP-hard, but approximations and heuristics work well in practice.
6. **The learning theory pipeline:** Probability $\rightarrow$ Concentration $\rightarrow$ Uniform convergence $\rightarrow$ Sample complexity $\rightarrow$ Algorithms. Week 1 covers the first two steps.

Confidence Check

Rate your understanding of Week 1's core concepts (1-5):

- ☐ I can apply Hoeffding's inequality to bound sample estimation error
- ☐ I understand the concentration hierarchy (Markov $\rightarrow$ Chebyshev $\rightarrow$ Hoeffding) and when each applies
- ☐ I can recognize convex functions and explain why convexity makes optimization easier
- ☐ I can use Big-O notation to describe algorithm scalability
- ☐ I understand how Week 1 tools connect to the PAC learning preview

If you rated yourself below 3 on any item, review that section. These concepts are foundational for the entire course.

Next week: We'll use Hoeffding + union bounds to prove that finite hypothesis classes are PAC learnable, then extend to infinite classes using VC dimension. Week 1's exponential concentration is what makes learning possible!